Recall that `.get()` only returns a single match. If there is no match or there are multiple matches, `.get()` raises an exception. Try querying an `id` that doesn't exist yet:

```
>>> Musician.objects.get(id=42)
Traceback (most recent call last):
  File "<console>", line 1, in <module>
  File "RiffMates/venv/lib/python3.12/site-packages/django/db/models/
➥ manager.py", line 85, in manager_method
    return getattr(self.get_queryset(), name)(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "RiffMates/venv/lib/python3.12/site-packages/django/db/models/
➥ query.py", line 650, in get
    raise self.model.DoesNotExist(bands.models.Musician.DoesNotExist:
➥ Musician matching query does not exist.
```

Or querying with `first_name="John"`, you get the following:

```
>>> Musician.objects.get(first_name="John")
Traceback (most recent call last):
  File "<console>", line 1, in <module>
  File "RiffMates/venv/lib/python3.12/site-packages/django/db/models/
➥ manager.py", line 85, in manager_method
    return getattr(self.get_queryset(), name)(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "RiffMates/venv/lib/python3.12/site-packages/django/db/models/
➥ query.py", line 653, in get
    raise self.model.MultipleObjectsReturned(
bands.models.Musician.MultipleObjectsReturned: get() returned more
➥ than one Musician -- it returned 2!
```

As my dad himself will admit, he doesn't belong in our musicians table; it is time to fix the second problem. The `.delete()` method on your object instance deletes the corresponding object from the database. Note that unlike the other database operations you've performed, this one is directly on the instance itself. Use the `joseph` value you queried before and delete it:

```
>>> joseph.delete()
(1, {'bands.Musician': 1})
```

The `.delete()` method returns a tuple containing the number of objects deleted and a dictionary with key–value pairs for the type and number of objects deleted. Most of the time, you just ignore the response from `.delete()`.

Calling `.delete()` does not change your object—it only affects the database. While the `joseph` object is within scope, you can still access all of its attributes. In fact, you can create a new entry in the database with the same values as the old one by calling `.save()` on the object. The value of the unique ID field will change, but all the other fields would be back. To see the effect of delete on the database, run a query looking for my non-musician father; you'll notice the entry is no longer there.

Your users aren't going to have access to your database, or they better not— that is just asking for trouble. To display your data, you need a view. The query methods you've learned so far can be implemented in views for display to your users.